7 SOLID dəyişiklikləri nəzarətdə saxlamaq üçündür

Kafe tətbiqi böyüyür. Yeni endirim növü, başqa qəbz formatı, başqa baza və yeni bildiriş kanalı əlavə olunur. Hər kiçik dəyişiklik bütün proqramı pozmamalıdır.

SOLID bu problemi düşünmək üçün beş dizayn prinsipinin baş hərfləridir. Dizayn burada kodun hansı hissəsinin hansı işi gördüyünü və hissələrin bir-birinə necə bağlı olduğunu müəyyən etməkdir.

Hərf	Əsas fikir
S	Bir-birindən ayrı dəyişmə səbəblərini qarışdırma.
O	Seçilmiş davranışı mövcud əsas axını dağıtmadan genişləndir.
L	Yeni növ köhnə müqavilənin vədini pozmasın.
I	İstifadəçini lazım olmayan metodlardan asılı etmə.
D	İş qaydasını dəyişən texniki detala möhkəm bağlama.

Bu cədvəli əzbərləmək hələ prinsipləri bilmək deyil. Növbəti səhifələrdə hər prinsipin hansı dəyişiklik problemini həll etdiyini görəcəyik.

Məqsəd: daha çox sinif və interface yaratmaq yox, dəyişikliyin hara təsir edəcəyini aydınlaşdırmaqdır.

S eyni sinfi niyə müxtəlif adamlar dəyişir

Bir InvoiceService qəbzin vergisini hesablayır, onu bazaya yazır, PDF görünüşünü qurur və email göndərir.

Sabah vergi qaydası dəyişir. Sonra dizayner qəbzin görünüşünü dəyişir. Sonra baza texnologiyası dəyişir. Hamısına görə eyni sinif dəyişirsə ayrı səbəblər bir yerə yığılıb.

SRP, Single Responsibility Principle, əlaqəli bir dəyişmə səbəbinə xidmət edən məsuliyyət sərhədi qurmağı istəyir.

Mümkün bölgü

```
InvoiceCalculator -> Meblegi ve vergini hesablayir.
InvoiceStore      -> Bazada saxlayir.
InvoiceRenderer   -> Qebzin gorunusunu hazirlayir.
InvoiceSender     -> Hazir qebzi gonderir.
```

İndi qəbzin rəngi dəyişəndə vergi hesabı dəyişməyə bilər. Baza dəyişəndə görünüş koduna toxunmaya bilərik.

Bu işləri ardıcıllaşdıran ayrıca xidmət də ola bilər: hesabla, saxla, göndər. Addımları əlaqələndirmək özü bir məsuliyyət ola bilər.

SRP “hər sinifdə bir metod olsun” demək deyil. Vergi hesablayan sinifdə bir neçə əlaqəli köməkçi metod normaldır. Əsas sual həmin metodların eyni səbəbə görə birlikdə dəyişib dəyişməməsidir.

Bir yerin içindəki hissələrin eyni məqsədə xidmət etməsinə **cohesion**, uyğunluq deyilir. SRP uyğun olmayan dəyişiklikləri ayırmaqla bunu yaxşılaşdırmağa kömək edir.

O yeni endirim əsas sifariş axınını dəyişməsin

Normal müştəriyə 10, VIP müştəriyə 20 faiz endirim veririk. Sonra tələbə endirimi əlavə olunur. Hər yeni növ üçün Checkout-un içində uzun seçim məntiqi artırsa əsas axın da daim dəyişir.

Endirim hesabını ayrıca müqaviləyə çıxaraq:

C#

```
public interface IDiscount
{
    decimal Calculate(decimal total);
}

public class VipDiscount : IDiscount
{
    public decimal Calculate(decimal total)
        => total * 0.20m;
}
```

`decimal` dəqiq onluq ədəd tipidir. `0.20m` həmin tipdə 20 faiz dəyəridir. `Calculate` endirimin öz məbləğini qaytarır; yekun ödəniş `total - endirim` olur.

Checkout konstruktorla IDiscount alsın. Yeni StudentDiscount eyni müqaviləni yerinə yetirsə Checkout-un hesab axını dəyişmədən yeni variantı istifadə edə bilər.

Bu **OCP**, Open Closed Principle, üçün nümunədir: seçilmiş dəyişiklik istiqamətində genişlənməyə imkan veririk, sabit əsas məntiqə az toxunuruq.

Qeydiyyat və seçim yerinin dəyişməsi normaldır. OCP “heç bir fayl heç vaxt dəyişməsin” demir. İki sabit hal üçün sadə `switch` də kifayət edə bilər; hər seçimə dərhal ayrıca pattern qurmaq məcburi deyil.

L yeni növ eyni vədi saxlamalıdır

Bir müqaviləmiz var: “etibarlı mətni verilən yerə yaz”. Proqram bu müqaviləyə güvənərək yazma metodunu çağırır.

İndi yalnız oxumağı bacaran obyekt həmin yazma müqaviləsini götürür və bütün çağırışlara “mən yaza bilmirəm” xətası verir. Adı və metod imzası uyğun olsa da davranış vədi pozulub.

LSP, Liskov Substitution Principle, yeni alt növün baza müqaviləsinin yerində istifadə ediləndə onu istifadə edən kodun düzgün gözləntilərini pozmamasını istəyir.

Müqavilə nəyi deyir

```
Qebul edilen giris: etibarli metn ve yazila bilen yol.
Ved edilen netice: metn yazilir ve ya ved edilen xeta gelir.

Yanlis yeni davranis:
    Etibarli girisde de "hec vaxt yaza bilmirem".
```

Çağırışdan əvvəl tələb olunan qaydaya **precondition**, uğurlu çağırışdan sonra vəd edilən nəticəyə **postcondition** deyilir. Obyektin etibarlı vəziyyətində qoruduğu qayda isə **invariant** adlanır.

Müqavilədə disk dolanda xəta ola biləcəyi deyilibsə, həmin xətanın baş verməsi təkbaşına LSP pozuntusu deyil. Əsas məsələ gözlənilməyən yeni məhdudiyyət və ya zəiflədilmiş vəddir.

Praktik həll: yalnız oxuyan obyektə yazma vədi yükləmə. Müqaviləni onun həqiqətən bacardığı əməliyyatlara uyğun seç. Növbəti prinsip bu bölgünü açır.

I hər istifadəçiyə lazım olan müqaviləni ver

Kafedə bir hesabat xidməti yalnız sifariş oxuyur. Amma ona verilən IOrderStore müqaviləsi oxuma, əlavə etmə, silmə, arxivləmə və başqa işlərlə doludur.

Hesabat xidməti istifadə etmədiyi yazma əməliyyatlarına da bağlı olur. Yalnız oxuyan başqa implementasiya isə yalandan yazma metodları təqdim etməyə məcbur qala bilər.

C# müqavilə nümunələri

```
public interface IOrderReader
{
    string Read(int id);
}

public interface IOrderWriter
{
    void Save(string order);
}
```

Hesabat IOrderReader alır. Yazma xidməti IOrderWriter alır. Ehtiyac varsa bir konkret sinif hər iki müqaviləni həyata keçirə bilər. `void` metodun nəticə dəyəri qaytarmadığını bildirir. Bu kiçik nümunədə sifariş mətnlə təmsil edilir.

Bu **ISP**, Interface Segregation Principle, üçün nümunədir. Burada “client”, müştəri, interface-dən istifadə edən kod deməkdir; kafedəki insan deyil.

ISP “hər interface-də yalnız bir metod olsun” demir. Birlikdə istifadə olunan oxuma metodlarını eyni müqavilədə saxlamaq normal ola bilər.

Nəticə: interface-i onu istifadə edən kodun real ehtiyacına görə böl. Sadəcə xırdalamaq üçün bölmək də əlavə qarışıqlıq yarada bilər.

D iş qaydası konkret baza sinfinə bağlanmasın

Sifarişi tamamlamaq biznes işidir. SQL bağlantısını açmaq isə həmin işin texniki detalıdır. Sifarişin əsas məntiqi birbaşa konkret SQL kitabxanasının təfərrüatına bağlanarsa texnologiya dəyişəndə əsas məntiq də dəyişə bilər.

İşin ehtiyacını bir müqavilə ilə göstəririk:

Asılılıq istiqaməti

```
Sifaris isi -> IOrderStore <- SqlOrderStore

Sifaris isi saxlamani teleb edir.
SqlOrderStore hemin telebi SQL ile yerine yetirir.
```

Bu **DIP**, Dependency Inversion Principle, üçün nümunədir. İş qaydası çox vaxt **high-level**, onu yerinə yetirən texniki detal **low-level** adlandırılır. Hər ikisini uyğun abstraksiya ilə bağlayırıq. Abstraksiya burada “sifarişi saxla” ehtiyacını ifadə edən müqavilədir.

DIP ilə DI fərqi: DIP dizaynda nəyə bağlı olduğumuzu seçir. DI isə seçilmiş asılılığı obyektə kənardan vermək üsuludur. Interface yaratmadan da constructor-la konkret SQL sinfi vermək mümkündür; bu DI-dır, amma texniki detaldan asılılığı aradan qaldırmır.

Qatların “controller service-i çağırır” formasında düzülməsi təkbaşına DIP zəmanəti deyil. Müqavilənin özü də konkret SQL texnologiyasının tipləri ilə doludursa detal yenə əsas işə sızmış ola bilər.

Beş prinsipi eyni dəyişiklik üzərində yoxla

Yeni qəbz formatı əlavə edirik. Özümüzdən bunları soruşaq:

S: görünüş dəyişdiyi üçün vergi və saxlanma siniflərini də dəyişirəmmi? Edirəmsə məsuliyyət sərhədi qarışmış ola bilər.

O: yeni qəbz implementasiyası əlavə edib mövcud sifariş axınını saxlayırammı?

L: yeni implementasiya eyni qəbul edilən girişlər üçün eyni vədi yerinə yetirirmi? Yoxsa köhnə kod indi əlavə xüsusi hallar yazmağa məcburdur?

I: qəbzi yalnız göstərən kodu çap və silmə kimi lazım olmayan əməliyyatlardan asılı etmişəmmi?

D: sifariş işi qəbz müqaviləsini bilir, yoxsa konkret PDF kitabxanasının iç quruluşunu?

Bir dəyişikliklə məşq

Əvvəl TextReceipt verən sifariş axını qur. Sonra PdfReceipt əlavə et. Sifariş kodunda hansı dəyişikliklərin lazım olduğunu yaz. Məqsəd interface sayını artırmaq yox, yeni variantın təsir etdiyi yerləri azaltmaqdır.

Yadda saxla: SOLID düzgün proqramı avtomatik yaradan düstur deyil. Dəyişiklik səbəbi, müqavilə və asılılıq istiqaməti barədə düşünmək üçün vasitədir. Əlavə abstraksiya problemi asanlaşdırmırsa onun xərci də nəzərə alınmalıdır.